Important contest instructions

Please read the following instructions carefully. They contain important information on how to name and write your programs, how to run them, and how to prepare them for submission.

NEW for 2026

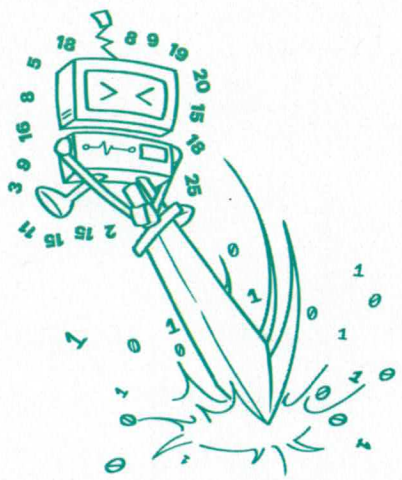
Problem numbers will start at 01 instead of 00.

Directory input changed. In the main `"files"` directory, there will be a subdirectory for each problem that requires reading from a directory. The subdirectory name will be specified in the problem statement and follows the `"probLL"` format, where LL is a **two-letter code** (e.g. AC).

Language versions

The judges will use the following versions for our supported languages.

Language	Version
Java	25.0.2
Python	3.14
C / C++	15.2.0



Java

Your program source file must be named **probNN.java** and your class name must be **probNN**, where **prob** is all lowercase and NN corresponds to the two-digit problem number (e.g. 03). Java programs may rely on the standard Java library.

- Example class name: `public class prob03`
- Example source file name: `prob03.java`

Python

Your program should be named **probNN.py**, where NN corresponds to the two-digit problem number (e.g. 03). Python programs may rely on the standard Python library.

C / C++

Your source file should be named **probNN.c** or **probNN.cpp**, where NN corresponds to the two-digit problem number (e.g. 03). You may use the glibc/libm libraries and the standard headers. Download the "C/C++ Developers Guide" from https://hpecodewars.org/downloads

Note

If the problem statement examples and the input/output files disagree, assume the files are correct. We will announce any corrections as needed.

Program input

Most problems require input (except Problem 01). You have two options: file input or keyboard input. Some problems may require directory input.

File input

Your program may read input from a file named **input.txt** for all problems. The file will be in the same directory as your program. Use `"input.txt"` as the filename and **do not prefix it with a path**.
Download the "Guide to data file I/O" from https://hpecodewars.org/downloads

Keyboard input

Your program may read input from standard input (the keyboard). There are two options to provide input to your program via standard input.

1. The preferred option is to redirect the contents of a file to your program's standard input.

```
java prob01 < input.txt
python prob01.py < input.txt
prob01.exe < input.txt
```

1. Otherwise you may type the input manually, or copy/paste it from the dataset files.

> Tip: Type `Ctrl-Z <enter>` to signal the end of input.

Directory input

For some problems, you may need to read from a directory named `"files"`. The directory will be in the same directory as your program. The student ZIP file includes the `"files"` directory. Be sure to **move the `files` directory** to the same directory as your program before you run it.

In the `"files"` directory, there will be a subdirectory for each problem that requires reading from a directory. The subdirectory name will be specified in the problem statement and follows the `"probLL"` format, where LL is a **two-letter code** (e.g. AC). Download the "Guide to Directory I/O" from https://hpecodewars.org/downloads

Program output

Your program must send the output to standard output (the default for any print/printf output). Do **not** include any prompts for input in your output! The only output for your program must be the expected output for the problem.

Program testing

Test your program with the Python Check Program script (`checkProgram.py`) in the student ZIP file. The script will automatically run your program with each dataset for that problem. See `README.txt` in the student ZIP file for more details.
We strongly encourage you to test your program with all datasets in the `student_datasets` directory of the student ZIP file!

Program submission

Your program must be submitted through the contest site using a text box (with a large character limit, don't worry!). You must copy and paste the **entire program** from your local source code file into the text box. Once you select the problem number and language from the respective drop-down menus, you can submit your program for judging.

problem 01

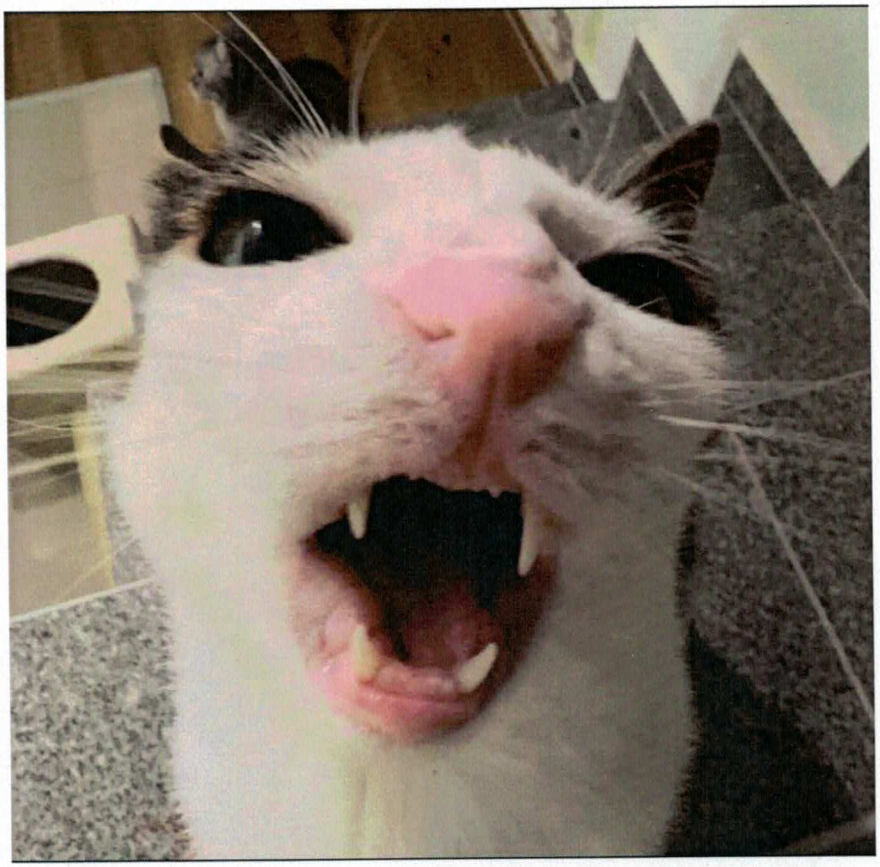
Say hello to Crunchy Cat

1 point

Welcome to HPE CodeWars!

A competitive coding competition can be scary for some ... but Crunchy Cat wants to try anyway.

Please welcome Crunchy Cat to the competition and wish her luck!



Input

There is no input for this, **and only this**, problem.

Output

Print out the following line to the standard output (the screen).

```
Welcome to HPE CodeWars, Crunchy Cat! Don't worry! It's going to be fun!
```

Discussion

This is one of the two problems allowed for early submission to the contest. The purpose of this problem is to make sure your coding environment is setup correctly, and that your team is able to create code which will output correctly to the standard output (the screen).

problem 02

6 7 or 41?

1 point

Nothing kills a meme faster than the adults picking it up, right?

6 7 is dead, long live 41!

Let's fix some memes!



Input

The first, and only, line of input contains a single "word" containing letters, numbers, and punctuation. It does not contain any spaces.

The maximum word size will be 25 characters.

```
67?Tired
```

Output

Print out the following sentence on a single line to standard output (the screen). Replace XYZ with the word you read from the input:

- Meme fixed? XYZ, Bet!

```
Meme fixed? 67?Tired, Bet!
```

Discussion

This is one of the two problems allowed for early submission to the contest. The purpose of this problem is to make sure your coding environment is setup correctly, and that your team is able to create code which can read and use data.

No decisions or complex coding is needed for this problem. You simply need to read in the "word" and then put it into the sentence. The only special handling you may need to deal with is the newline character, depending on your coding language.

Additional Examples

Input 2	Output 2
41-Better	Meme fixed? 41-Better, Bet!

problem 03

YadaYadaYada: Epizeuxis

1 point



Repeating something two to three times can emphasize its importance: "I need it, I need it, I need it!" is more emotional than one alone.

Epizeuxis (e-pi-zook'-sis) is the repetition of words or phrases without intervening words. *(See?! CodeWars puzzles can be educational, too!)*

These are some classic examples:

- *Hamlet:* "Words, words, words."
- *MacDuff:* "Oh, horror, horror, horror!"
- *McConaughey:* "Alright, alright, alright!"

Teach your computer to repeat itself.

Input

Read one line of input. It may have numbers, spaces, letters, and punctuation characters.

```
Yada.
```

Output

Repeat the line three times on a single line. Do not add spaces, but keep any found in the input.

```
Yada.Yada.Yada.
```

Discussion

Test with all datasets to be sure you manage various punctuation.

Additional Examples

Input 2	Output 2
`Beetlejuice!`	`Beetlejuice!Beetlejuice!Beetlejuice!`

Input 3	Output 3
`Say "Epizeuxis" three times.`	`Say "Epizeuxis" three times.Say "Epizeuxis" three times.Say "Epizeuxis" three times.`

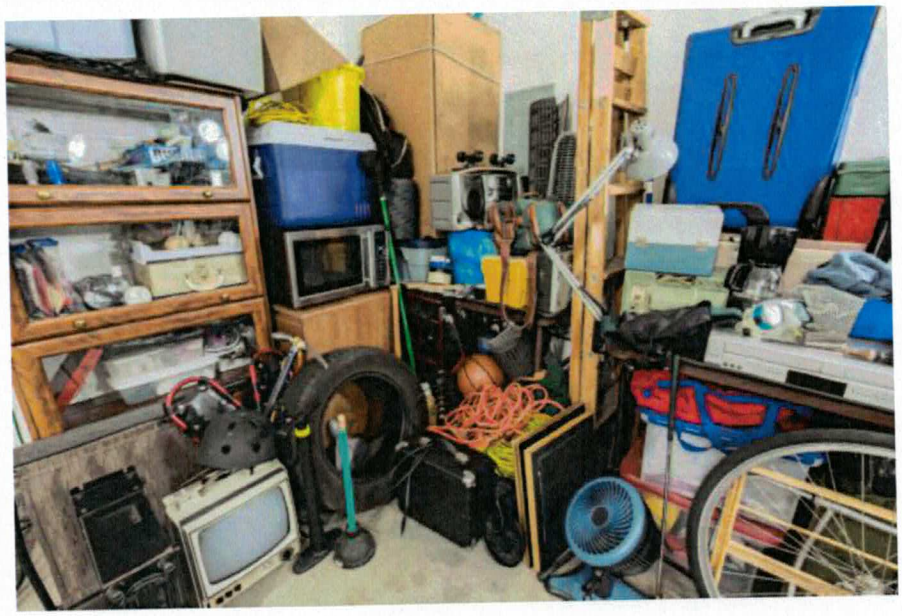
problem 04

Order More

1 point

Storage is growing into the future. People can't seem to get rid of stuff, but they still need a place to put their stuff. Units4U™ Storage Systems is ready to help!

We are continually setting up additional storage so we can help everyone! Simply enter the number of storage units you need, and we will get them ready!



Input

Read a single positive integer (from 1 to 1 million), representing the number of storage units requested.

```
6
```

Output

From past history, we know that everyone orders fewer units than needed, and we have learned to anticipate their additional call when they ask for more room, so we need to let our reservation department know how many actual storage units to find and keep ready. Today, the number of actual units needs to be twice the number requested, so double the request and print that result so we can send the right value for reservations.

```
12
```

Additional Examples

Input 2	Output 2	Input 3	Output 3	Input 4	Output 4
987654	1975308	642975	1285950	25	50

problem 05

Bop Bit

1 point

You find a game of Bop It while sifting through a neighbor's garage sale.

It is a fast-paced electronic handheld game where players must quickly follow escalating voice and visual prompts to manipulate various inputs (like Bop, Twist, and Pull) in the correct sequence. If you miss a cue or hesitate, the game ends and your score reflects how long you lasted.

You think that you can automate a program to make it so you can play forever and get a really really high score. Your program uses a single bit to execute the instructions of the device.



Input

The first line of input will contain the one word instruction. The second line of input will contain the single bit value.

```
Twist
0
```

Output

If the instruction says Bop, you take the input and turn it into a 0.
If it says Pull, you take the input and turn it into a 1.
If it says Twist, you take the input and flip the bit. (ex. 1 to 0, or 0 to 1)

```
1
```

Additional Examples

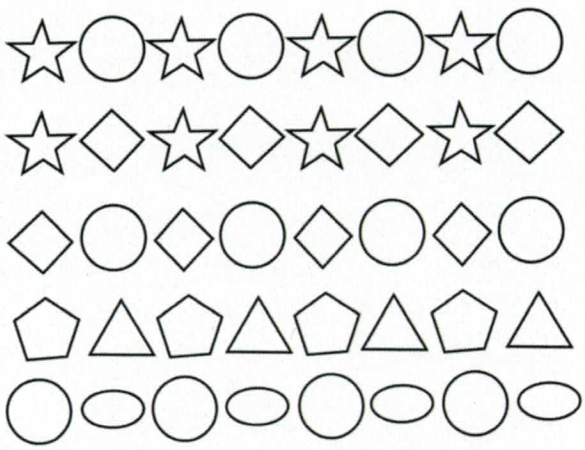
Input 2	Output 2	Input 3	Output 3
Bop 1	0	Pull 0	1

problem 06

Every Other

2 points

Our EnCodeWars team has been finding new ways to encode messages. One repeats characters 3 times too often, and one expands the distance from the first comma to the next period. Um? One uses hieroglyphs (really?). Unfortunately, decoding instructions weren't included. Luckily, we've managed to work out the technique for a few of the notes and need your help to restore the original text.



In the messages we recovered, the first half of the original text is spread out to every other character, then the second half is inserted. Please provide each restored message to our DeCodeWars team.

Input

Read one line of text, which may hold letters, numbers, and punctuation.

```
Sektihpopdi,n gb ucth airta cctaenr sb em aqyu ibtee  ae fsfiemcptliev em!
```

Output

Print the characters in the odd positions, then print the characters in the even positions. The first character is in position 1, the second in position 2, etc.

```
Skipping characters may be a simple method, but it can be quite effective!
```

Additional Examples

```
Input 2  (Just to prove you can read all characters.)
AmBnCoDpEqFrGsHtIuJvKwLxMyNzO P1Q2R3S4T5U6V7W8X9Y0Z  !a@b#c$d%e^f&g*h(i)j,k.l

Output 2
ABCDEFGHIJKLMNOPQRSTUVWXYZ abcdefghijklmnopqrstuvwxyz 1234567890 !@#$%^&*(),.
```

```
Input 3
Ssoemlee smse stseaxgte.s  (mAaByG UiVnAtTe ngtbi ofnrarl luyr ekre.e.p. )u

Output 3
Some messages may intentionally keep useless text. (ABGUVAT gb frr urer...)
```

PN-3-0 EO-18-6 EO-0-1 EO-16-0 EO-22-0 EO-23-0 EO-24-0 EO-25-0 EO-26-0

problem 07

Will It Build?

2 points

Once you write more complex programs you will have external code (libraries) that your program depends on. In order to use these dependencies you need to specify a version number to ensure compatibility. This is often done using a system called Semantic Versioning (SemVer).



In SemVer, version numbers are in the format **MAJOR**.*Minor*.patch. The rules are as follows:

- Increment the **MAJOR** version when you make incompatible changes.
- Increment the *Minor* version when you add functionality in a backwards-compatible manner.
- Increment the patch version when you make backwards-compatible bug fixes.

Your task is to determine if a recently published package version is newer, older, or identical to the version you require.

Input

The first line of input will contain a number indicating the number of test cases. Each test case consists of two versions separated by a space: the required version and the recently published version. Both versions will be in MAJOR.MINOR.PATCH format.

```
4
1.0.0 1.0.1
3.2.1 3.2.1
0.9.9 1.0.0
10.2.0 9.2.0
```

Output

For each test case, output the recently published version and whether it is "newer", "older", or "identical" compared to the required version.

```
1.0.1 is newer
3.2.1 is identical
1.0.0 is newer
9.2.0 is older
```

Additional Examples

Input 2	Output 2	Input 3	Output 3
6 2025.9.2 2025.9.12 2025.11.24 2025.12.24 12.2025.0 10.2025.1 99.1.1 100.0.0 16.0.1 16.2.0 1.99.5 1.100.2	2025.9.12 is newer 2025.12.24 is newer 10.2025.1 is older 100.0.0 is newer 16.2.0 is newer 1.100.2 is newer	5 99.87.65 100.1.2 99.20.99 99.5.89 123.4.56 123.4.56 9.8.7 9.8.8 10.1.2 10.1.0	100.1.2 is newer 99.5.89 is older 123.4.56 is identical 9.8.8 is newer 10.1.0 is older

problem 08

Typing Tutor

2 points

Mico Pilot is doing his best to learn to type. The Typing Tutor program provides a single line to practice and he types it several times.

Unfortunately, Mico Pilot seems to keep making mistakes. The program counts how many times he made a mistake and reports the results.



Input

The first line is the text Mico Pilot should duplicate. All the following lines are Mico Pilot's attempts to type. When Mico Pilot is done, he types a single period.

```
The quick brown fox jumps over the lazy dog.
The quick brown fox jumps over the lazy dog.
The quick brown fox jumped over the laxy dog.
The quick brown dog jumps over the lazy fox.
The quick brown fox jumps over the lazy dogs.
This is really hard.
The quick brown fox jumps over the lazy dog
The quick brown fox jumps over the lazy dog.
.
```

Output

Each line should match the first line exactly. After reading the first line, print every line that has an error. Also, count how many lines had an error, and print the total when done.

```
The quick brown fox jumped over the laxy dog.
The quick brown dog jumps over the lazy fox.
The quick brown fox jumps over the lazy dogs.
This is really hard.
The quick brown fox jumps over the lazy dog
Number of incorrect lines: 5
```

Additional Examples

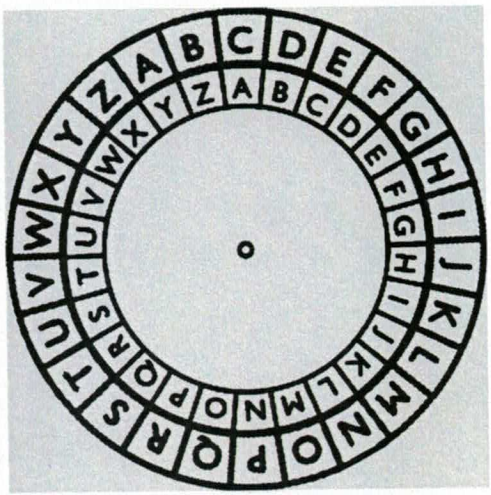
Input 2	Output 2
My name is Mico Pilot. My name is Mico Pilot. My name is Mico Pilot. .	Number of incorrect lines: 0

problem 09

Shifting Letters

3 points

Our EnCodeWars cipher enthusiasts have been "working" long hours to find ways to hide messages. After weeks of work, they believe the first method they found to be quite devious. (*Spoiler. Not so much.*) They decided that numbers and punctuation rarely provide the full meaning in a conversation, so only the letters in the messages need to be changed. They chose to rotate all the letters so the words are unreadable.



With a single integer at the beginning of a transmission, the receiver will know enough about how to restore the original message. The designers are confident no other listener will learn how to use that number.

Use your DeCodeWars talents to see if their design can be broken by restoring each message.

Input

Read two lines of input. The first is an integer N (from 0 to 10000), the rotation distance to move the letters. The second line is the encoded text. It may have numbers, spaces, letters, and punctuation characters.

```
109
XjyzRvmn 29 dn di ocz tzvm 2026.  (Cjkzapggt, tjp'mz vgmzvyt vrvmz ja ocdn!)
```

Output

Convert only each letter by rotating it to the letter N steps later in the alphabet. Lowercase remain lowercase, and uppercase remain uppercase. Whenever you reach Z/z, continue at A/a. Only modify letters.

```
CodeWars 29 is in the year 2026.  (Hopefully, you're already aware of this!)
```

Discussion

Rotating 13 letters is known as ROT-13, which has the added benefit that it can decode its own encoding.

Additional Examples

Input 2	Output 2
13 Aha envy errs bar gnat. Nun rail reef one tang!	Nun rail reef one tang. Aha envy errs bar gnat!

Input 3	Output 3
10000 10000 Bydkdsyxc coow k vsddvo exxomocckbi.	10000 Rotations seem a little unnecessary.

PN-5-0 SL-7-1 SL-3-4 SL-7-1 SL-0-0 SL-10-0 SL-22-0 SL-23-0 SL-7-1 SL-15-1 SL-7-1

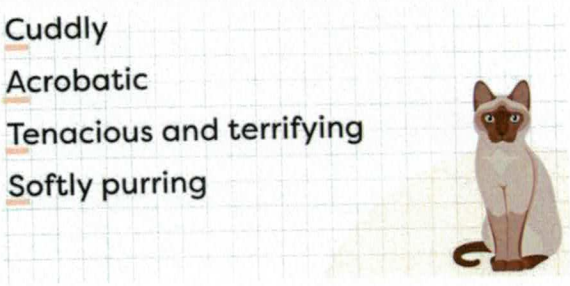
problem 10

Book Cipher

4 points

In a Book Cipher, a message is encoded into numbers using a specific book. Each word or letter in the message is encoded by counting to words and letters on a page of the book.

Since only the sender and receiver know the book, it is very difficult to break such a code. For example, in the picture, an encoded word-letter code for "Cats" could be 1-1 2-6 6-4 3-9.



This is one of the family of EnCodeWars puzzles this year. We will send you several encoded messages based on our shared book. Prove your ability to join the DeCodeWars effort, and decrypt them all.

Book Pages

There are 3 pages in our book. They are each very short, but they can encode many messages. The COMPLETE book is shown below.

The first page is part of a poem from Lord of the Rings. The second page shows the first words of the Gettysburg Address. The third page is Random Text.

```
Lord of the Rings (LR) page:
One ring to rule them all, one ring to find them,
One ring to bring them all and in the darkness bind them

Gettysburg Address (GA) page:
Four score and seven years ago our fathers brought forth,
upon this continent, a new nation,

Random Text (RT) page:
The quick brown fox, jumps over the lazy dog. Wars in 2026 have many En-Code-ings.
29! = 29*28*27! ABCde FGHIJ KLMno pqrst UVWXY 123456789?
```

Input

There is one line of input. Each element in the line is of the form XX-W-C. "XX" is a letter code, identifying the page in our shared book. "W" is an integer, the number of the word on the page. "C" is an integer, the number of the character in the word. Words and characters are counted starting from 1. Each element is separated from the next by a space.

```
Input 1: RT-15-4 LR-7-1 LR-22-4 GA-13-7 RT-10-0 RT-12-3 RT-18-2 RT-16-3
```

```
Input 2: GA-1-0 GA-8-0 LR-18-0 LR-7-0 RT-4-0 LR-10-0 LR-7-0 RT-9-0 RT-23-3 LR-3-2
GA-15-3 RT-9-4
```

```
Input 3: RT-18-8 GA-0-0 RT-18-3 RT-0-1000 RT-24-1 RT-24-4 LR-0-0 RT-17-0 RT-24-9
RT-24-8
```

See the next page for Output and detailed decoding instructions!

Book Cipher

continued

Output

To decode each element of the cipher:

1. Reference the correct book page.
2. Count to word W on that page.
3. Count to character C in the word, and print it.

For example, let's say we have to decode "RT-4-2":

1. "RT" is the "Random Text" page that begins with "The quick brown fox, jumps".
2. Words are separated by spaces and line breaks. All connected characters are a single "word". Word 4 on the RT page is "fox," (including the comma.)
3. Character 2 in "fox," is "o".

In summary, RT-4-2 is the "o" in "fox," and we would print "o".

There are **two special cases:**

1. If W is 0, simply print a space. Ignore C.
2. If C is 0, print the entire word followed by a space.

```
Output 1: CodeWars 29!
```

```
Output 2: Four fathers and one fox, find one dog. Wow.
```

```
Output 3: 7 * 14 = 98
```

Discussion

In your `files/probAI` folder, you will find "book_cipher_pages.txt" with the "LR", "GA", and "RT" pages. Your program does NOT need to read the file. It is provided so you may copy the text of the book pages into your program.

History and Variations

Book ciphers were used during the American Revolution and World War II. They can be easily modified, such as by numbering lines, or by using a memorized song or poem. As long as only the sender and receiver know the source, they can be impossible to break. But they can be easily implemented against any text. For example, if this packet of CodeWars problems is our "book", we could write a Page-Word-Letter cipher (see below) as a cipher for "9. Code".

```
PN-9-0 BC-4-1 BC-9-4 BC-8-7 BC-4-5  -->  9. Code
```

Explanation: On the "Phone Numbers" page, '9.' (with the period) is the 9th "word". PN-9-0 tells us to print the entire word and a space. In the first words on the "Book Cipher" page, BC-4-1 is the 'C' in 'Cipher', BC-9-4 is the 'o' in 'into'. BC-8-7 is the last 'd' in 'encoded'. BC-4-5 is the 'e' in 'Cipher'.

Read up on Book Cipher options when you next need to hide a message.

problem 11

Bit Flip Probability

4 points

You can't always be sure that your data is correct. Every bit has a chance of being incorrect. In a small string of bits, the chance of having a bit flip is small. As a string of bits gets longer and longer, the chance of at least one error occurring increases more and more. At some point, the likelihood of an error is too high. Please help determine when the given string has passed that threshold.

Input

The first line of input contains 3 space-separated decimal values:

- probability of bit flip from 0 to 1 (chance of a 1 being incorrect)
- probability of bit flip from 1 to 0 (chance of a 0 being incorrect)
- threshold for acceptable error (entire string)

The second line contains a message string up to 32 bits in length. The minimum message length is 1 bit.

```
0.003 0.005 0.030
10000011
```

Output

First output the probability of the message being correct, truncated to the thousandths place. On the second line, output whether the string of bits fits within the acceptable error threshold. If so, output "accept", else output "reject".

```
0.966
reject
```

Discussion

Don't round or truncate until the very end! Rounding intermediate values can mess up your results.

Subtract the probability of a 1 being incorrect from 1 to calculate the probability of a correct 1. Do the same calculation for the probability of a correct 0. Subtract the threshold for acceptable error from 1 to calculate the threshold probability of the message being correct.

$$correct1 \qquad correct0 \qquad thresh_{MESSAGE}$$
$$1 - 0.003 = 0.997 \qquad 1 - 0.005 = 0.995 \qquad 1 - 0.030 = 0.970$$

Message:

10000011

Multiply the probability of each individual bit being correct to calculate the probability of the total message being correct.

$$0.997 * 0.995 * 0.995 * 0.995 * 0.995 * 0.995 * 0.997 * 0.997 = 0.966497820$$

Compare the probability of the total message being correct with the threshold probability of the message and choose whether to accept or reject the message.

$$0.966 < 0.970 \qquad reject$$

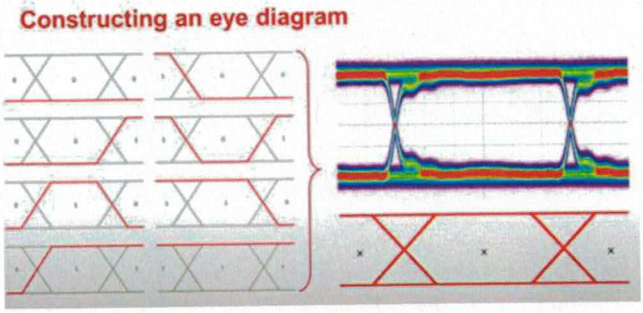
Since our calculated probability of an accurate message is worse than our threshold, we must reject the message.

problem 12

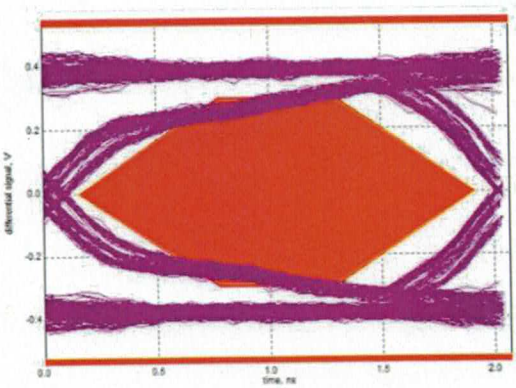
It's in my eye!

4 points

Eye diagrams are a way to evaluate the data quality of a high-speed digital signal. Engineers use an oscilloscope to capture eye diagrams. The data signal is probed and measured for a period of time to capture multiple instances of every possible bit transition. The image to the right titled "Constructing an eye diagram" shows the process of how the eye is formed. The size of the eye tells the quality and can be used identify any issues such as jitter, noise, or signal reflections. Typically, the larger the eye is, the better the signal integrity is.



An eye mask is a predefined region that is used as a keep out region as seen in the image to the right. When a point in the eye diagram is found inside the mask that would be considered a mask violation. For a digital system to pass an eye diagram test the system must have no eye mask violations.



Eye Mask

```
- - - - - -
- - O O - -
- O O O O -
- O O O O -
- - O O - -
- - - - - -
```

We will use a simple mask, as seen on the left. The locations with O represents the Eye Mask, where we don't want to see any signal.

Each '-' is an allowed location for the signal. If you would like to copy this into your solution, you can find EyeDiagram.txt in your `files/probBQ` folder.

Input

The input files for this question will contain a 6x6 table similar to the eye. Every "-" is where the signal was seen during testing. Every "X" is where the signal was never seen. You are only concerned with the signal "-" seen; you can ignore any "X".

Output

If the signal ("-") was never seen in the Mask ("O" locations), just print "PASS". If **any** signal is found in a Mask location, print "FAIL", and reprint the input, marking the invalid "-" locations using an exclamation mark "!".

Input 1	Output 1	Input 2	Output 2	Input 3	Output 3
	PASS		FAIL		FAIL
- - X - - -		- - - - - -	- - - - - -	- - X - - -	- - X - - -
- - X X X -		- - - X X -	- - ! X X -	- X X X - -	- X X X - -
- X X X X X		- X X X X X	- X X X X X	X X X X X -	X X X X X -
- X X X X X		- X X X X X	- X X X X X	X X - X X -	X X ! X X -
- X X X X -		- - - X X -	- - ! X X -	- - X X - -	- - X X - -
- - - - - -		- - - - - -	- - - - - -	- - - - - -	- - - - - -

Discussion

Input 1 shows a wide area where the signal was never seen. Since the signal was never within the mask region ("O" locations), this is a PASS. In Input 2 we see that it FAILS because we found 2 "-" inside our eye mask . There are some "X" found outside the mask but that is ok for this test. Input 3 has a "-" inside the Mask. Therefore this input fails our test. We mark where the dash violates our region using an exclamation mark "!".

The input examples above were shifted down one line to help with visual comparison on this page. Your input datasets do not have a blank line at the top.

problem 13

How Long Will It Take?

4 points

The amount of hours in a day is fixed at 24. There's only so much time in a day to get things done. That's why many audio and video players offer the ability to speed up or slow down playback.



If you listen to a 1 hour podcast at 1.5x speed, it will only take 40 minutes to finish it. Your task is to determine how long it will take to play a media file at a given speed.

Input

The first line of input will be the playback length of the media file in hours and minutes (h:mm format). The minimum playback length is 0:10 (10 minutes). The following lines will each contain a playback speed as a decimal number (e.g., 1.0x for normal speed, 0.5x for half speed, 2.0x for double speed). The input will end when the playback speed is 0.0x. The playback speed will range from 0.1x to 10.0x inclusive.

```
1:30
1.2x
1.5x
2.0x
0.0x
```

Output

For each playback speed, output the original playback length and the adjusted play time in h:mm format. Truncate fractional minutes; do not round. The adjusted play time will be less than 10 hours. Include the playback speed in the output.

```
1:30 is 1:15 at 1.2x speed
1:30 is 1:00 at 1.5x speed
1:30 is 0:45 at 2.0x speed
```

Additional Examples

Input 2	Output 2	Input 3	Output 3
1:00 1.0x 2.0x 1.5x 3.0x 1.3x 0.0x	1:00 is 1:00 at 1.0x speed 1:00 is 0:30 at 2.0x speed 1:00 is 0:40 at 1.5x speed 1:00 is 0:20 at 3.0x speed 1:00 is 0:46 at 1.3x speed	2:45 0.5x 1.5x 2.5x 0.8x 0.0x	2:45 is 5:30 at 0.5x speed 2:45 is 1:50 at 1.5x speed 2:45 is 1:06 at 2.5x speed 2:45 is 3:26 at 0.8x speed

problem 14

Mirrored Text

5 points

The Mirror of Erised is malfunctioning. The only way to get it to behave is to write a message backwards and then read its reflection in the mirror.



Input

Input will be multiple lines of text up to 100 characters in length per line. END on a line by itself ends the input.

```
Hogwarts is threatened!
Man the boundaries, protect us!
Do your duty to our school!
Minerva McGonagall.
END
```

Output

Reverse each of the words in each line, while preserving everything else about it (spacing, word order, punctuation, etc.). Words are separated by spaces. Hyphenated words should be treated as whole words. **Hyphens and apostrophes are parts of words**. The only other punctuation will be: commas, periods, question marks and exclamation marks. Those should all be kept in their place, not reversed with neighboring words.

```
strawgoH si denetaerht!
naM eht seiradnuob, tcetorp su!
oD ruoy ytud ot ruo loohcs!
avreniM llaganoGcM.
```

Discussion

Punctuation will always be touching the end of a word. You will not encounter more than one punctuation per word. Do not count hyphenated words or words with apostrophes as punctuated, treat them as a letter in the word. See Additional Examples 2 and 3.

Mirrored Text

continued

Additional Examples

Input 2	Output 2
Do you mean to tell me you could've taken your hand outta that cuff at any time? No. Not at any time. Only when it was funny. Bl-bl-bl-! C'mon, Eddie! Where's your sense of humor? Is he always this funny or only on days when he's wanted for murder? END	oD uoy naem ot llet em uoy ev'dluoc nekat ruoy dnah attuo taht ffuc ta yna emit? oN. toN ta yna emit. ylnO nehw ti saw ynnuf. -lb-lb-lB! nom'C, eiddE! s'erehW ruoy esnes fo romuh? sI eh syawla siht ynnuf ro ylno no syad nehw s'eh detnaw rof redrum?

Input 3	Output 3
Emergency. Induction. Port. --Tali END Nothing after "END" should be printed.	ycnegremE. noitcudnI. troP. ilaT--

problem 15

FiboNotchi Locks

5 points

In a Fibonacci series, each element is found by adding the prior two. The first elements of the standard Fibonacci series are:

1 1 2 3 5 8 13 21 34 55 89 144 ...

If you join (remove spaces from) all the values in the standard Fibonacci series, you get:

1123581321345589144...

If you choose two different values to start the series (like: 12 3), you create a new Not-Fibonacci (or FiboNotchi) series:



```
12 3 15 18 33 51 84 ...
1231518335184...
```

Our EnCodeWars team decided to make sure our workspace is secure and provided FiboNotchi hints to the lock combinations. Please help us get our snacks by decoding their instructions.

Input

The input is one line of five positive integers less than 100, separated by spaces. The first two start the series. The final three integers A,B,C are in increasing order (A < B < C).

```
1 1 3 11 15
```

Output

Create the series. Each newly added value is equal to the sum of the prior two. Put them all together, omitting spaces. Report the digit found at each location of the last three integers, separated by one space. The '1 1' input above creates: 1123581321345589144...

IMPORTANT: The first character is position 1 (not position 0).
That is, the values at locations 1,2,3,4,5,6,7,8,9,10,11,12,13,14,15 are: 1,1,2,3,5,8,1,3,2,1,3,4,5,5,8.
So for the provided Input above, the digit at position 3 is '2', at 11 is '3' and at 15 is '8'.

```
2 3 8
```

Additional Examples

Input 2	Output 2	Input 3	Output 3
12 3 7 60 80	8 4 8	3 4 55 66 99	4 0 1

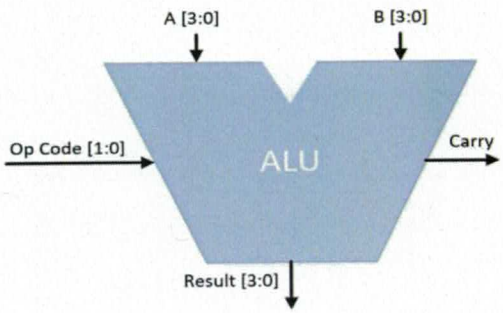
PN-6-0 PN-1-1 PN-3-1 PN-4-1 PN-3-1 PN-5-0 FL-1-1 FL-10-6 FL-1-2 FL-3-4 FL-4-3 FL-4-2 FL-0-0 PN-2-0

problem 16

Basic Math

5 points

An Arithmetic Logic Unit (ALU) performs basic digital arithmetic and bitwise operations in a computing system. The picture to the right shows a simple Arithmetic Logic Unit (ALU). In the case of this ALU, it takes 2 4-bit operands (A and B), performs the operation specified by the 2-bit Operation code (Op Code), and outputs a 4-bit result.



For the purposes of this problem, assume all numbers (A, B, Result) are positive integer values. Below are two tables. The left one shows the binary encoding of the opcodes. The right shows a truth table for the different op codes.

Note that the addition of 1 + 1 results in the result carrying over into another bit position (10, aka 2 in decimal). If this carry over happens in the most significant bit (aka leftmost), the answer cannot fit on the Result bus, and the Carry output is set. For Example: 1000 + 1001 = 1_0001 (8 + 9 = 17). But 16 cannot fit in 4 bits, so the ALU will report a Result of 0001 (the lowest 4 bits of the addition) and a carry of 1.

OPERATION	OPCODE[1:0]
ADD	00
OR	01
AND	10
XOR	11

A	B	ADD RESULT	OR RESULT	AND RESULT	XOR RESULT
0	0	0	0	0	0
0	1	1	1	0	1
1	0	1	1	0	1
1	1	10	1	1	0

Input

Read one line of input. It will have A, B and OpCode in that order and in binary, separated by spaces

```
0001 1000 00
```

Output

Simulate the ALU operation and output the result as a 4-bit binary number, followed by the carry bit. Remember that the result bus is only 4 bits wide. Here we added 1 + 8 = 9, which fits in 4 bits (1001), so the carry is 0.

```
1001 0
```

Discussion

Note that because of the bitwise nature of AND, OR and XOR, the carry bit is always 0 for these operations.

Additional Examples

Input 2	Output 2	Input 3	Output 3	Input 4	Output 4
1100 0001 01	1101 0	1010 1001 10	1000 0	0101 1001 11	1100 0

Input 5	Output 5	Input 6	Output 6
1011 1011 00	0110 1	1111 1111 00	1110 1

problem 17

Sudoku Verifier

5 points

Sudoku is a popular number puzzle played on a 9x9 grid. The objective is to fill the grid so that each row, each column, and each of the nine 3x3 subgrids contain all of the digits from 1 to 9.

Your task is to verify if a given Sudoku solution is valid.

		1						
						6	4	9
7	9		5			1		8
2	8			9		7		
					7			
5	7			6		9		
9	6		7			5		2
						8	9	4
		8						

Input

The input will consist of a completed Sudoku puzzle with some ASCII formatting to help visualize the subgrids. Each line will represent a row of the Sudoku grid, with numbers separated by spaces. Horizontal lines (using equals and plus signs) and vertical bars will be used to delineate the 3x3 subgrids.

```
|=======================|
| 1 2 3 | 4 5 6 | 7 8 9 |
| 4 5 6 | 7 8 9 | 1 2 3 |
| 7 8 9 | 1 2 3 | 4 5 6 |
|=======+=======+=======|
| 2 3 4 | 5 6 7 | 8 9 1 |
| 5 6 7 | 8 9 1 | 2 3 4 |
| 8 9 1 | 2 3 4 | 5 6 7 |
|=======+=======+=======|
| 3 4 5 | 6 7 8 | 9 1 2 |
| 6 7 8 | 9 1 2 | 3 4 5 |
| 9 1 2 | 3 4 5 | 6 7 8 |
|=======================|
```

Output

If the Sudoku puzzle was solved correctly, output CORRECT. Otherwise, output INCORRECT followed by the rows, columns, and subgrids which are incorrect. Row and column numbering starts at 1, and subgrid numbering starts at 1 from the top-left subgrid moving right and then down. If there is more than one error per row, column, or subgrid, only list it once. If the grid is INCORRECT, but all rows (or columns, or subgrids) are correct, print a 0 for that section.

```
CORRECT
```

Additional Examples

Input 2

```
|=======================|
| 1 2 3 | 1 5 6 | 7 8 9 |
| 4 5 6 | 7 8 9 | 1 2 3 |
| 7 8 9 | 1 2 3 | 4 5 6 |
|=======+=======+=======|
| 2 3 4 | 5 6 7 | 8 9 1 |
| 5 6 7 | 8 9 1 | 2 3 4 |
| 8 9 1 | 2 3 4 | 5 6 7 |
|=======+=======+=======|
| 3 4 5 | 6 7 8 | 9 1 2 |
| 6 7 8 | 9 1 2 | 3 4 5 |
| 9 1 2 | 3 4 5 | 6 7 8 |
|=======================|
```

Output 2

```
INCORRECT
Rows: 1
Columns: 4
Subgrids: 2
```

Input 3

```
|=======================|
| 1 2 3 | 4 5 6 | 7 8 9 |
| 4 5 6 | 7 8 9 | 1 2 3 |
| 7 8 9 | 1 2 3 | 4 5 6 |
|=======+=======+=======|
| 2 1 3 | 4 5 6 | 7 8 9 |
| 5 4 6 | 7 8 9 | 1 2 3 |
| 8 7 9 | 1 2 3 | 4 5 6 |
|=======+=======+=======|
| 3 2 4 | 4 5 6 | 4 5 6 |
| 6 5 7 | 7 8 9 | 1 2 3 |
| 9 8 1 | 1 2 3 | 7 8 9 |
|=======================|
```

Output 3

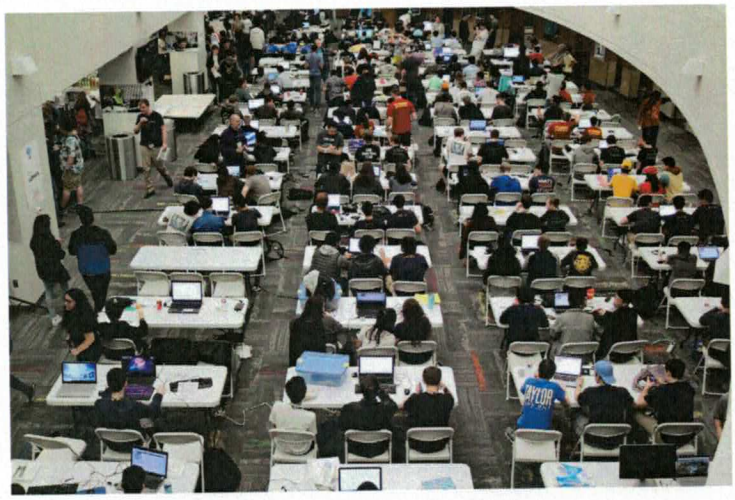
```
INCORRECT
Rows: 7 8 9
Columns: 2 3 4 5 6 7 8 9
Subgrids: 0
```

problem 18

Team Numbering

6 points

Based on the incredibly sad true story ...



When teams arrive at a CodeWars event, the desks are conveniently in numerical order. But we want teams from the same school to not sit together, to help reduce the desire to lean over and share information.

Unfortunately, assigning team numbers has annually proven to be a difficult task. Somehow, each year's attempt at solving the "Team Numbering Problem" leads to failures, and some manual intervention is required.

The Discussion section on the next page describes our latest team numbering algorithm. Please implement it and test it against several sample datasets and let us know where it fails.

Input

The first line is an integer N, the count of schools. The next N lines have a single letter to identify the school, and an integer count of that school's team count. Some locations for CodeWars can accept more than 3 teams per school, and we like to admit as many students as we can fit.

```
5
A 3
B 2
C 4
D 3
E 3
```

Output

Print the total Team Count. Then reprint the input for each school, adding on each line a colon, then the assigned team numbers **in order of assignment**. If any two of a school's team numbers differ by 1, also add the word FAILURE at the end of that school's line.

```
15
A 3: 103 106 109
B 2: 112 115
C 4: 102 105 108 111
D 3: 114 101 104
E 3: 107 110 113
```

Discussion

The **floor()** function finds the integer portion of a number. floor(3)=3. floor(4.67)=4.
101 will be the lowest team number. [Below, bracketed text shows values for the above input.]

- Find the Total Team Count "totalTc". maxTeamNumber = 100+totalTc. [totalTc=15, maxTeamNumber=115]
- Find the Max Team Count for any school "maxTc". [maxTc=4]
- The widest separation of teams will be: skipVal = floor(totalTc/maxTc). [skipVal=floor(15/4)=3]
- If skipVal equals 1, make skipVal=2. One school's numbering might fail, but others will work out.
- baseTeamValue = 100 + skipVal. [baseTeamValue=103]
- nextTeamNumber = baseTeamValue. [nextTeamNumber=103]
- Loop through each school's Team Count:
- Assign to the next school team the teamNumber = nextTeamNumber [first assignment is 103]
- Add skipVal to nextTeamNumber [After first assignment, nextTeamNumber is 106]
- if nextTeamNumber > maxTeamNumber:
    - baseTeamValue = baseTeamValue - 1 [When this is first reached, we move from 103 to 102]
    - nextTeamNumber = baseTeamValue

There are many ways we can improve this algorithm. Compare dataset 3 to dataset 4 to see how team ordering can fix a failure. Hopefully, as long as no school brings over half of the total teams, this algorithm will keep each school spread out.

Additional Examples

Input 2	Output 2
4 J 3 K 3 L 5 M 4	15 J 3: 103 106 109 K 3: 112 115 102 L 5: 105 108 111 114 101 M 4: 104 107 110 113

Input 3	Output 3
3 S 2 T 5 U 2	9 S 2: 102 104 T 5: 106 108 101 103 105 FAILURE U 2: 107 109

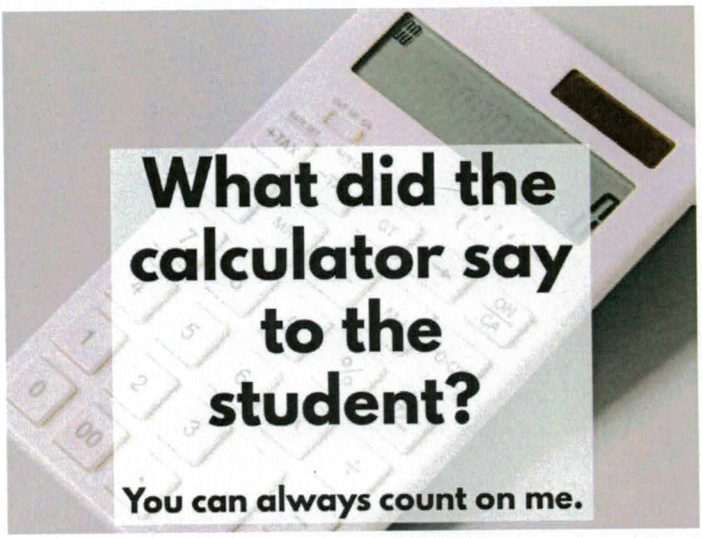
problem 19

Hello PEMDAS my old friend

6 points

Where's a calculator when you need one? Quick, help the student you are mentoring understand how the order or operations must work in math calculations.

As a gentle reminder, PEMDAS stands for Parentheses, Exponents, Multiplication and Division, Addition and Subtraction which is a mnemonic for the order of operations to follow when evaluating a mathematical expression.



Input

You will receive, on a single line, a mathematical problem (using only integers) to solve using the order of operations: PEMDAS. There will be no parentheses or exponents in your input.

```
10*3+4-8/2+2*100-18+3-6
```

Output

Carry out the operations in PEMDAS order, working left to right. On each line of your output carry out one operation following PEMDAS order.

Remember: Multiplication and Division have the same "level" as do Addition and Subtraction. Operations on the same level should be performed working left to right. Do not move down a "level" until all operations at the current level have been carried out.

Your last line should be the final answer which shows the output of your calculations following PEMDAS order of operations.

```
30+4-8/2+2*100-18+3-6
30+4-4+2*100-18+3-6
30+4-4+200-18+3-6
34-4+200-18+3-6
30+200-18+3-6
230-18+3-6
212+3-6
215-6
209
```

Hello PEMDAS my old friend

continued

Additional Examples

Input 2	Output 2
222-10/5+33*9-2+88/4+6/2+9-1*1	222-2+33*9-2+88/4+6/2+9-1*1 222-2+297-2+88/4+6/2+9-1*1 222-2+297-2+22+6/2+9-1*1 222-2+297-2+22+3+9-1*1 222-2+297-2+22+3+9-1 220+297-2+22+3+9-1 517-2+22+3+9-1 515+22+3+9-1 537+3+9-1 540+9-1 549-1 548

Michael Carter Wallace

Hello

problem 20

Phone Numbers

7 points

1. 2. 3. 4. 5. 6. 7. 8. 9. 0.

A company may want a way to remember its phone number, or at least figure it out.



Our EnCodeWars team started a free (Can you pay in confusion?) consulting business to help. Consider the local recycling center's phone number 143-586-0729. They've assigned a different letter to each digit, and the first 2 parts of the phone number add up to the third. They tell customers: "THE+OLD=CANS! and THE=143, and you can work out the rest!"

And the nearby pet supply store uses "DOG-CAT-VIBE" as their encoded phone number, and it also adds up: "DOG+CAT=VIBE, CAT=786".

We had hoped our EnCodeWars team would keep their annoyances internal. Please help us keep the public sane. Use some DeCodeWars logic to determine the encoded phone numbers.

Input

Read 2 lines of input. The first is the letter-encoded phone number. The second line is a list of 3 or 4 known letter-to-digit assignments in the format "A=1 B=2 ...".

```
DOG-CAT-VIBE
A=8 C=7 T=6
```

Output

Solve the sum: First3 + Second3 = Last4, using the given letter=number assignments, such that all 10 digits are used. Use the output format shown.

```
DOG-CAT-VIBE = 249-786-1035
```

Discussion

Run against all datasets. The provided letters may not always be in the same word. Remember the 4-digit number could start with a zero.

Additional Examples

Input 2	Output 2
THE-OLD-CANS H=4 T=1 E=3	THE-OLD-CANS = 143-586-0729

Input 3	Output 3
OUR-DAY-TIME O=6 T=1 U=2 Y=9	OUR-DAY-TIME = 624-879-1503

Input 4	Output 4
NEW-CAR-TOYS A=5 C=7 W=9	NEW-CAR-TOYS = 849-753-1602

PN-1-0 SL-1-1 SL-2-2 SL-3-5

problem 21

Proper Pallet Packing

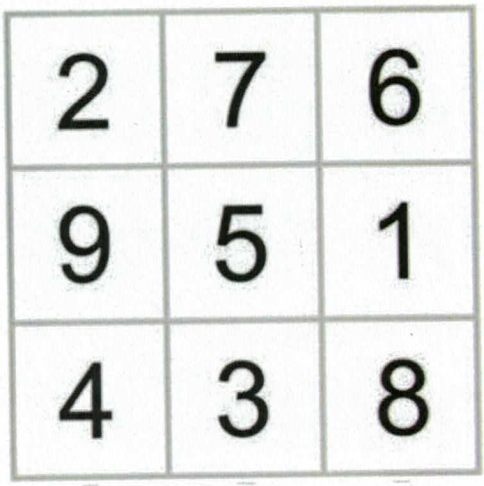
7 points

Our packing people are perfectionists. They pack each pallet with packages of persimmons and popcorn and peanuts, to permit a perfect balance. That is, each package in each portion of the pallet has different number of pounds, but the proper sum of each row, column, and diagonal is magically perfect (always the same sum!)

Possibly, when overnight packing ends with partially packed pallets, the morning packers must puzzle out the pounds for each portion of the pallet to complete the picture.

Given a few of the packages on the 3x3 pallet, pack the remaining parcels with the proper pounds such that all eight sums are predictably perfect.

To protect our pallets, all packages for this puzzle range from 1 to 99 pounds.



Input

The input is 3 lines of 3 integers from 0 to 99, separated by spaces. Single digits have an additional leading space.

```
 0  9  0
 3  0  0
 8  0  0
```

Output

Complete the magic square. A positive number is a packed parcel that must remain present. A zero is a package for which you need to pick the pounds. Replace all zeros with a unique number from 1 to 99. Print the solved grid. Place one space between each number, and one additional space before single digits.

```
 4  9  2
 3  5  7
 8  1  6
```

Discussion

Remember each package must be from 1 to 99 pounds, with no duplicates. There is only one solution.

Additional Examples

Input 2	Output 2	Input 3	Output 3	Input 4	Output 4
2 0 50 0 0 0 0 0 0	2 95 50 97 49 1 48 3 96	0 0 25 0 0 0 0 34 10	28 4 25 16 19 22 13 34 10	0 59 53 0 0 7 0 0 0	17 59 53 79 43 7 33 27 69

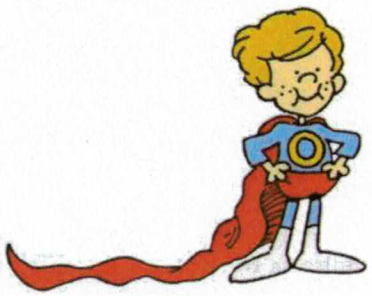
problem 22

Farewell Problem 0

7 points

CodeWars numbering has finally decided to give up counting like an array and move to the rest of the world.

Problem 0 no longer exists. But of course, we recognize that 0 is still important and this puzzle is meant to show just how much we still respect 0 and want to honor it.



Input

The input is one line with 2 integers, N < M, separated by a space, each from 1 to 100 million.

```
1 100
```

Output

Count how often a 0 is seen in all numbers from N to M, including both. The answer may be 0. See? Respect! Repeat the input, then a colon and space, then the count of zeros.

```
1 100: 11
```

Discussion

See below - zero is used over 68 million times in all numbers thru 100 million. It's still important.

A loop to 100 million could take a long time. Your program must finish within 10 seconds. Consider ways to speed up the search. For example, how many zeros appear from 1000 to 1999, and how often does such a range appear?

Additional Examples

Input 2	Output 2	Input 3	Output 3	Input 4	Output 4
1 100000000	1 100000000: 68888897	4321 4329	4321 4329: 0	9991 12000	9991 12000: 1603

11

111 +

1100

10000

10010

problem 23

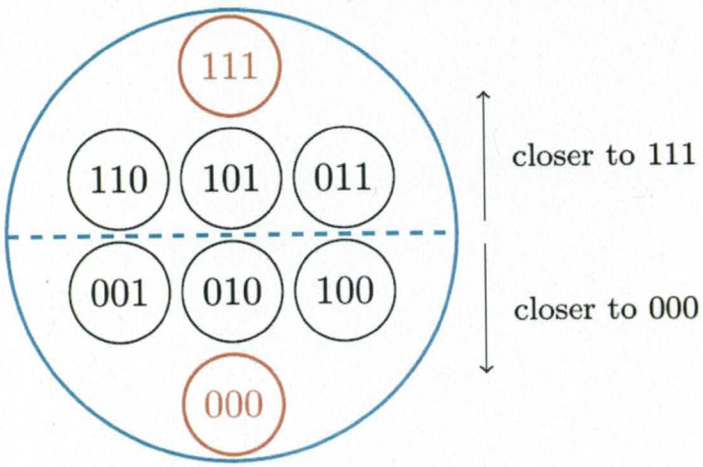
3 Bit Repetition

7 points

When a string of 1 and 0 digits is sent, a single incorrect digit can invalidate the output. To prevent this, we send each digit multiple times.

A three-bit repetition is a SECDED (single error correcting, double error detecting) code. It can either correct one bit flip or detect two bit flips. We send our 1 and 0 digits using SECDED to protect the full string.

Decode 3-bit repetition codes that may contain bit flips. If a digit cannot be decoded within acceptable error thresholds, mark it with an X.



Input

The first line contains four space-separated decimal values:

- Probability of bit flip from 0 to 1 (chance a received 1-bit is incorrect)
- Probability of bit flip from 1 to 0 (chance a received 0-bit is incorrect)
- Threshold for acceptable error per 3-bit repetition (maximum error for identifying a digit)
- Threshold for acceptable error in entire output (maximum error for identifying ALL digits)

The second line contains the received 3-bit strings for our 0 and 1 digits, which may include bit flips.

```
0.003 0.005 0.010 0.020
110 010 000 111
```

Output

On the first line, output each identified digit as 0 or 1. For digits beyond the acceptable error threshold, output X.

Multiply the probability of accuracy for all digits. On the second line, output this probability truncated to the thousandths place. If any digit was marked X, this value should be 0.000.

On the third line, print "accept" if the probability meets the acceptable threshold, otherwise print "reject".

```
1001
0.989
accept
```

Discussion

Assume that at least two 1-bits in a 3-bit string represents a 1-digit, and at least two 0-bits represents a 0-digit. Therefore 111, 110, 101, and 011 represent a 1; and 000, 001, 010, and 100 represent a 0.

See the next page for more discussion including the four possible bits-to-digit probabilities.

Discussion (continued)

Note: The probability values in this section are specific to the input, repeated here for your convenience. Your program must calculate probabilities for each specific input dataset, which will differ from this example.

```
0.003 0.005 0.010 0.020
110 010 000 111
```

We use the bit probabilities to determine the accuracy probability for each 3-bit string.
$P1_{incorrect}$ = 0.003 and $P0_{incorrect}$ = 0.005 are the first two values from the input.
$P1_{correct}$ = 0.997 and $P0_{correct}$ = 0.995 are calculated as 1 minus the incorrect probabilities.

We use Bayes' Rule: If you know the probability of A and B each happening independently, and only one can occur, then the probability of A is:

```
ProbA / (ProbA + ProbB)
```

Reminder: Do not truncate any probability values until the final output. In all four cases below, we assume at least two bits are correct.

We assume two 1-bits (011, 101, 110) decode to a 1-digit:

Probability 110 was actually a 0-digit: `P110was0 = (.003)(.003)(.995) = 0.000008955`
Probability 110 was actually a 1-digit: `P110was1 = (.997)(.997)(.005) = 0.004970045`
Probability we are correct: `PTwo1sIs1 = P110was1 / (P110was0 + P110was1) = 0.998201446`

We assume two 0-bits (001, 010, 100) decode to a 0-digit:

Probability 010 was actually a 0-digit: `P010was0 = (.995)(.003)(.995) = 0.002970075`
Probability 010 was actually a 1-digit: `P010was1 = (.005)(.997)(.005) = 0.000024925`
Probability we are correct: `PTwo0sIs0 = P010was0 / (P010was0 + P010was1) = 0.991677796`

We assume three 0-bits (000) decode to a 0-digit:

Probability 000 was actually a 0-digit: `P000was0 = (.995)(.995)(.995) = 0.985074875`
Probability 000 was actually a 1-digit: `P000was1 = (.005)(.005)(.005) = 0.000000125`
Probability we are correct: `PThree0sIs0 = P000was0 / (P000was0 + P000was1) = 0.999999873`

We assume three 1-bits (111) decode to a 1-digit:

Probability 111 was actually a 0-digit: `P111was0 = (.003)(.003)(.003) = 0.000000027`
Probability 111 was actually a 1-digit: `P111was1 = (.997)(.997)(.997) = 0.991026973`
Probability we are correct: `PThree1sIs1 = P111was1 / (P111was0 + P111was1) = 0.999999973`

Final answer

Each 3-bit string must identify its digit with a probability greater than: $P_{DigitThreshold}$ = 1-0.010 = 0.990.

To be **accept**ed, the full string must have a probability greater than: $P_{AllDigitThreshold}$ = 1-0.020 = 0.980.

In this example, all four 3-bit strings identify their digit with probability greater than $P_{DigitThreshold}$ (0.990), so all digits are acceptable.

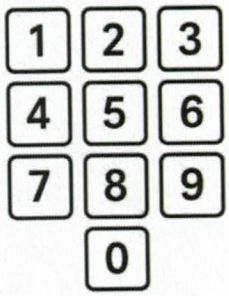
The product of all digit probabilities is 0.989, which exceeds $P_{AllDigitThreshold}$ (0.980), therefore we **accept** the result.

problem 24

Code Strings

7 points

Our CodeWars interns were emptying our snack cupboard, so we installed a digital lock. The lock uses a different length code each day without repeated digits.



The interns created ChatCWI ("Chat CodeWars Interns") to generate all the possible codes into one Efficient Code String (ECS), but they didn't check the results, and the ECS always has one digit wrong. So it is missing codes and instead holds duplicate or invalid codes. Help them restore the ECS to include all of the possible codes for the lock. The daily snack will be your reward.

Input

The first line contains three integers: **N** (the number of digits in each code, 2 <= N <= 4), **L** (the lowest digit available, 1 <= L <= 7), and **H** (the highest digit available, L < H <= 9).

The second line contains the corrupted ECS (less than 125 characters).

```
2 1 4
1413142324341
```

The string should contain all possible 2-digit codes using digits 1 thru 4: 12, 13, 14, 21, 23, 24, 31, 32, 34, 41, 42, 43 (12 codes total). But 12 and 21 are missing, and 14 and 41 are repeated.

Output

Output a single line in the format: The [wrong_digit] at position [pos] should be replaced with a [correct_digit]. The first character is at position 1.

```
The 4 at position 2 should be replaced with a 2.
```

In the example, position 2 contains '4' but should contain '2'. The correct string "1**2**13142324341" contains all 12 valid codes.

Additional Examples

Input 2

```
3 2 6
234235236243245246253254256326345346364356426436526546536456 23
```

Codes 354, 543, 635 are missing. Duplicate and invalid codes are found: 364, 636, 643.

Output 2

```
The 6 at position 38 should be replaced with a 5.
```

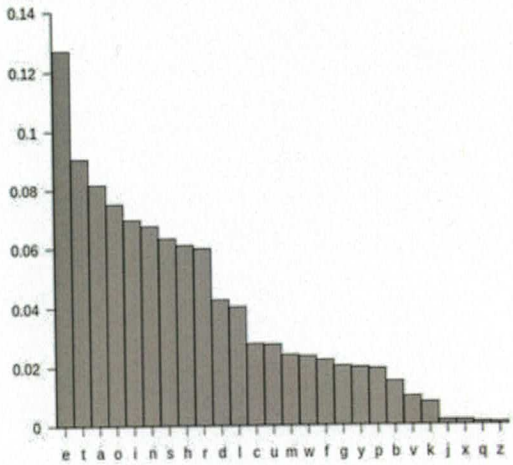
The correct string "234235236243245246253254256326345346**5**4356426436526546536456 23" contains all 60 valid 3-digit codes.

problem 25

Cryptogram Counting

8 points

In a cryptogram, each character (like 'E') that is in the first quote has a match with a different character (like 'C') at the exact same location in the second quote. Perhaps every 'C' in the first quote may have its match with 'R' in the second. In the example below, you can see that each 'E' becomes 'C', 'C' becomes 'R', and 'R' becomes 'E'. Usually, every letter is different from its original, but they could be the same, like every 'D'.



The expected frequency of each letter in the English language can be used to help decode cryptograms. You can see from the Frequency Chart picture, the most frequent letters are ETAOINS... It's not surprising that the most frequent letters include 4 vowels, since they're needed in every word.

Our EnCodeWars designers created one-line cryptograms to solve, but they are too short to follow the expected Frequency Chart, so we introduce the "Puzzle Frequency Chart" (PFC). Count the use of each letter (*for reference, in the example, see each letter's total count below itself. The resulting letter frequency is the same in both the original and encrypted message, because the letter locations are the same.*) Order the letters, starting with the most frequent. If multiple letters have the same frequency, place them in the order they first appear in the message. Below, E appears 4 times. 'I' and 'S' both appear 3 times, but 'S' is first in the quote, so 'S' is listed first in the PFC. 'E,S,I' in the original message correspond to 'C,Z,S' in the encrypted message.

```
CodeWars is incredibly awesome! -> RxdcAuez sz skrecdsnwo uaczxgc!
22242223 33 3122423111 2243212     22242223 33 3122423111 2243212
ESICODWARNBLYM                     CZSRXDAUEKNWOG
```

Input

Read two lines of input. The first line is the **PFC-O**: Puzzle Frequency Chart for the Original unencrypted message.
The second line is the encrypted message.

```
ESICODWARNBLYM
RxdcAuez sz skrecdsnwo uaczxgc!
```

Output

Create a **PFC-E**: Puzzle Frequency Chart for the Encrypted message. Use Uppercase only. Print it. Then use it to decrypt every letter to the letters in the PFC-O, and print the original message. Keep the same upper/lower case as in the encrypted message. Any numbers, spaces or punctuation should not change.

```
CZSRXDAUEKNWOG
CodeWars is incredibly awesome!
```

Cryptogram Counting

continued

Discussion

Have you run your program against every dataset? Notice that if you use an output dataset as input, it will recreate its own input file! (Just a suggestion to test you've got it right.)

Additional Examples

Input 2	Output 2
NTPUEALOKMBRSI Heyo: u3od pnn 5dp(35, hrma3ti & drh3yr@yc0h"!	HYDROPNEUMATIC Note: k3ep all 5pa(35, numb3rs & pun3tu@ti0n"!

Input 3	Output 3
SEHLAIDBYTOR Dse dram dse deccd derdseccd bl ise derdsnge.	DESCRAMBLING She said she sells seashells by the seashore.

PN-4-0 CC-5-0 CC-8-0 CC-14-0 CC-15-0 CC-16-0 PN-17-0 PN-19-0 CC-12-0 CC-16-0

problem 26

Rock Reckoning

9 points



At the end of CodeWars Camp (it could happen...), as a final project, the camp teams had to work together. Since there were plenty of rocks nearby, the CodeWars Camp Counselors devised a way to both clear out the garden and address the overflow of energy in the campers.

Campers on the Mound Movers team repeatedly grabbed one rock and placed it in a pile specific to that camper. They worked synchronously so that each of their piles always held the same number of rocks.

When the Mound Movers stopped (for a required water break), the Summing Solvers worked as a group to count all the rocks remaining in the garden.

Oddly, the campers didn't lose their energy and really enjoyed the activity. They collected the data, threw all the rocks back in the garden (sigh), then started again with a different number of Mound Movers each time. For each session, the Summing Solvers provided their sum, and the Mound Movers provided the number of piles they made. Wait, what? Why didn't they count the number of rocks in each pile? (sigh, again.)

You are on the Programming Plodders team. Knowing the sum (S) of rocks remaining in the garden, and the number of mounds (M) for each session, determine the minimum number of rocks in the garden.

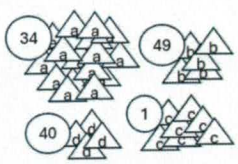
Input

The input is one line of numbers. The first is N, the number of counting sessions (from 1 to 8).
Then 2 numbers (S M) for each of the N sessions: S is the Sum of remaining rocks after M Mounds were gathered. Commas separate the values between sessions, as below.

```
4, 34 15, 49 5, 1 7, 40 4
```

Output

Determine the smallest total T of rocks which results in all of the session results. T will be less than 10 Million. For example, above, (T-34) is divisible by 15. (T-49) is divisible by 5, etc., Or, looking at the diagram to the right, the same T rocks create each result:
T = 34+15a = 49+5b = 1+7c = 40+4d. (a,b,c,d are positive integers.)




```
64
```

Discussion

The mounds all hold at least 1 rock. Don't compare ratios among the sessions; the teams may move at different speeds in each session. In Input 3, they were very fast: only one rock left each time for over 9 million rocks in the garden. Input 4 shows that CodeWars Camp is very popular.

Additional Examples

Input 2	Output 2
3, 11 6, 8 3, 9 10	29

Input 3	Output 3
8, 1 2, 1 3, 1 5, 1 7, 1 11, 1 13, 1 17, 1 19	9699691

Input 4	Output 4
1, 5111111 222224	5333335

problem 27

House Hunting

10 points

When Summer Or'der was growing up, she lived on a street with only three houses, numbered 1,2,3. Summer lived in house 3. She appreciated that the sum of the other house numbers equaled hers (1+2 = 3).



Later, she moved to house number 4 in a row of houses numbered 2,3,4,5. This was also interesting because the house numbers before hers added up to the house number after hers (2+3 = 5).

In her next house, this happened again, where her house was number 6 in houses numbered 1 thru 8 (1+2+3+4+5 = 7+8).

In her current house, she lives at house number 8 in a row numbered from 4 to 11, and now including her house, (4+5+6+7+8 = 9+10+11). Summer has decided she will hunt for houses where her house number is between (possibly included in) 2 sets of neighboring houses with the same sum. She realizes it may take a while to find such neighborhoods, but she'll still go house hunting. She'd like your help to determine the house numbers needed to achieve this.

Input

Each line of input is an integer N, Summer's potential house number. Help her discover if there is a neighborhood she would like that includes N as her house number.
When N=0, stop. Otherwise N can be from 3 to 10000.

```
8
6
204
13
10
9996
0
```

Output

For each line of input (N), search for the smallest set of sequential integers (fewest houses) that includes N, such that you can split them into two sets with the same sum, above and below N. N may be included in one or neither of the sums. The largest house number may be higher than 10000.

For each line, reprint N followed by a colon and a space. Then, if identical sums exist, print "M houses. A:B and C:D each have a sum of S." Replace each variable as shown below. However, if identical sums cannot be found, instead print "No neighborhood exists."

```
8: 8 houses. 4:8 and 9:11 each have a sum of 30.
6: 5 houses. 4:6 and 7:8 each have a sum of 15.
204: 57 houses. 174:203 and 205:230 each have a sum of 5655.
13: 7 houses. 9:12 and 13:15 each have a sum of 42.
10: No neighborhood exists.
9996: 11996 houses. 1999:9995 and 9997:13994 each have a sum of 47958009.
```

See the next page for Discussion and diagrams of example neighborhoods!

Discussion

A helpful formula: The sum of the integers from 1 to K is K(K+1)/2.

Important: Find **THE SHORTEST LINE OF HOUSES**. That is, find house numbers from A to B, and C to D such that **D-A is a minimum**, and sum(A to B) = sum(C to D), and exactly one of these 3 options is true:

- `N=B, at the end of the first sum.  (N=B = C-1)`
- `N=C, at the start of the second sum.  (B+1 = N=C)`
- `N is directly between the two sums.  (B+1 = N = C-1). N is not in either sum.`

Below is the house numbering for three of the examples in the Input on the prior page. These are the smallest neighborhoods (fewest houses) that create identical sums as requested.

When N=6, N is **at the end of the first sum** (A to B).

4	5	6	7	8
A		N=B	C	D

When N=13, N is **at the start of the second sum** (C to D).

9	10	11	12	13	14	15
A			B	N=C		D

When N=204, N is **directly between the two sums**.

174	175	...	202	203	204	205	206	...	229	230
A				B	N	C				D

problem 28

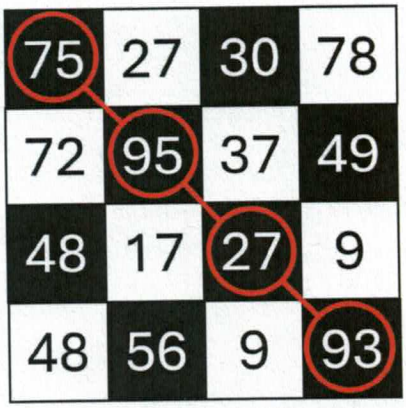
Maximize the Sum of the Square

10 points

King Chess has a map showing him how many gold coins are hidden in each square of his garden. Unfortunately, he is being pursued, so he must cross the garden quickly, gathering coins as he runs.

King Chess may enter the garden at any square in the first row. Then he must move to one of the three squares below the first (directly below, diagonally left, or diagonally right, like a chess king crossing down the board.)

Find the path through the garden that provides the largest sum of gold coins. For example, after entering the second row at 95, his move to the third row can be to 48, 17 or 27, and 27 leads to the best total.



Input

You'll be given a square array of positive integers of dimensions up to 50 x 50. The first line will be the dimension N of the square. Next will be N rows of N positive integers.

```
4
75 27 30 78
72 95 37 49
48 17 27  9
48 56  9 93
```

Output

On one line, print the maximum sum of any valid path, a colon, and the value of each of the N squares along the path from top to bottom, separated by single spaces.

```
290: 75 95 27 93
```

Discussion

In the above example, 290 gold pieces are found by moving along the diagonal: 75 + 95 + 27 + 93.

Note that in the large dimensions (like 50x50), generating each sum individually will not be fast enough. You'll need to be more clever than that.

Additional Examples

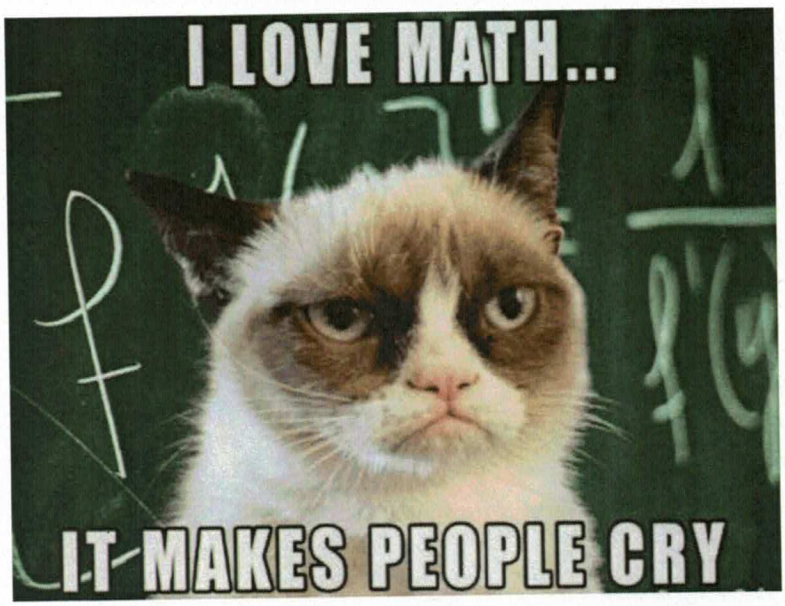
Input 2	Output 2
6 88 22 39 59 20 56 21 56 18 78 25 13 11 12 13 14 89 98 88 77 66 55 44 33 65 12 77 22 11 10 14 16 12 34 56 78	392: 59 78 89 55 77 34

problem 29

Algebra

10 points

So, we meet again ... math.



Input

You will receive two lines of input. The first will contain an algebraic equation. The second will give a variable you need to isolate on one side of the equation using basic algebra.

```
7w = 4y + 6g - 8
g
```

Output

Isolate terms in the following order, always adding terms to the other side of the equation to the right most side. Whatever term the input wants isolated should be isolated on whatever side of the equation it started. E.G. do not try to move the isolated term to the left like many textbooks would have you do.

Carry out all subtraction isolation steps before doing addition isolation steps. You will only have to perform a single multiplication or division step once, to fully isolate the target variable. IMPORTANT: Before doing the final multiplication or division, put parentheses around the opposite side.

Show each step of your process. Work left to right, only performing one operation per output line. All output should be spaced as shown, with a single space between all operators and terms.

```
7w - 4y = 6g - 8
7w - 4y + 8 = 6g
(7w - 4y + 8) / 6 = g
```

Discussion

In the example given on the previous page, there is only one term we can subtract to perform our isolation on 'g', and that is the 4y term. So we subtract that from both sides, which removes it from the right side, and adds it to the right most part of the left side.

Next in our order of operations is addition. We have one term we can remove from the side we are trying to isolate via addition, the -8. So, we add 8 to both sides, which removes it from the right side, and adds it to the right most part of the left side.

For division, we again only have one term which can be isolated via division, which is the 6g. Before division, we add parentheses around the terms on the left side. Then we divide both sides by 6, removing the 6 multiplication from the right side, and dividing the entire left side by 6, giving us our final answer to isolate g, which is: (7w - 4y + 8) / 6.

We did not carry out a multiplication step in the example, as that factor did not come up in our equation.

Do not combine like terms in your final answer. Once you get the term with the variable we want isolated, you may have to perform a final multiplication or division step, depending on how that term is arranged. For that final step, surround the terms opposite the isolated step with parenthesis.

For your algorithm, don't forget to take into account leading terms which are negative.

Additional Examples

Input 2	Output 2

`42z + 5a - 8 = 7t + 1`
`z`
	`42z - 8 = 7t + 1 - 5a`
	`42z = 7t + 1 - 5a + 8`
	`z = (7t + 1 - 5a + 8) / 42`

problem 30

Macaw Mapping

12 points



In our CodeWars bird sanctuary, we have some beautiful Macaws. We let them roam freely within the structure each day. To help us find the birds, we've created the Macaw Management Mapper (MMM) to track them. It also provides a handy map for any visitors to find the birds.

We've discovered the Macaws are very solitary, so only one is ever found in each section of the square grid. To avoid irritating the birds, the only active sensors are in unoccupied sectors. Each sensor reports the number of birds in neighboring sectors (in up to 8 surrounding squares.)

Knowing the number of birds in the sanctuary, and given sensor counts from various sectors, identify the squares which hold a Macaw.

Input

The first line holds 2 integers. The first is N, the size of the square grid map of the sanctuary (from 4 to 8). The second is the number of Macaws to be mapped. The next N lines provide a map of the NxN sanctuary. A number represents a sensor reading in an empty square, showing the count of neighboring squares with birds. A period marks an unknown square.

```
4 6
.2..
....
5..0
....
```

Output

Reprint the map, marking "M" for squares with Macaws. It is possible there are Macaws not detected by sensors, but the sensor values will let you identify the location of all Macaws. For example, in the output below, the top-right square must hold the 6th Macaw even though it wasn't detected by a sensor.

```
.2.M
MM..
5M.0
MM..
```

Additional Examples

Input 2	Output 2
7 10 1..... .3..22. 2.1.... ....22. ......3 ..2.2.. ...1.1.	 1.M.MM. M3..22. 2M1.... ....22M ...M.M3 ..2.2.M ..M1.1.

Note: The output shown here is shifted down one line to help with visual comparison on this page. Your output should not begin with a blank line.

problem 31

Buzzword Bingo

13 points

You and your team need to pay attention to a long webcast from a marketing consultant. To keep you occupied, you're playing buzzword bingo. Each of you have a card with 24 words arranged in a 5x5 grid (the middle square is a freebie). Words are arranged on the card left-to-right, top-to-bottom.

You need to scan the transcript and when you've got a bingo, five in a row, column or diagonal, shout bingo and then read off the results.

FINAL	MODEL	ATTIC	APART	UNITY
ESSAY	IDEAL	LIVID	GRATE	LEASH
CLEAR	STEER	*	SMITE	HASTY
TODDY	AMISS	PINCH	SKULL	QUEUE
KAPPA	GRUNT	MANGY	WHALE	CORAL

Input

The sets of bingo cards are in files in your files/probAD directory. So are the transcripts for each game. Your input includes the filename of both the card and game file to use.

```
files/probAD/cards.0 files/probAD/game.1
```

Each card file begins with the number of cards in the file (from 6 to 20), followed by a line with 24 5-letter words for each card. (Lines wrap in the example below, but in your datasets they will not).

```
6
VIGIL GUARD PALER BUYER MOOSE RADAR BLESS SHAPE NUTTY BONUS BUNCH LUMPY GIPSY VOGUE SNOUT
STOUT AIDER BRUNT GOLEM EVENT SOUTH MIGHT KNELT LEAPT
ANNOY OMBRE BILLY REPAY SUMAC SLANT JUNTO DOWNY EERIE ROTOR WOOZY WILLY ELDER DUNCE RHINO
TENSE BONUS TONAL BERRY ALIKE DROWN EPOCH SPLAT THIGH
FINAL MODEL ATTIC APART UNITY ESSAY IDEAL LIVID GRATE LEASH CLEAR STEER SMITE HASTY TODDY
AMISS PINCH SKULL QUEUE KAPPA GRUNT MANGY WHALE CORAL
WRONG PRUNE KITTY MATEY CREST CREAK SLYLY SPACE UNDER DINGY APPLY RHYME BOUND ADORE BARON
HOVER BADGE ANGRY ULTRA GODLY OPIUM FIZZY DIMLY SHOUT
MUCKY ELOPE BISON YOUTH BRIDE DREAD TENTH SNEER CLUCK GROUP HEDGE FLAKY BUSED SLEET MUSKY
STAMP WHINE EMBER LADLE SQUAD SHOVE MAPLE DONUT GAMUT
LEANT REFIT BELLE SLANT CAUSE AMONG BEAST GUILT AUGUR RABID CLOUT FOIST SHINY HOIST THIEF
OUGHT ICING EVENT OAKEN COULD WEAVE OXIDE DUSKY CLUMP
```

Each game file is a long list of 5-letter words separated by spaces and line-breaks. Words are called left to right starting the first line in the file. Here are the first couple of lines of a game file:

```
ROOST VERSE ABLED FLACK FLASK YIELD LUMPY WHEEL ENDOW SONIC LOWER USUAL CRIED
KHAKI PRIOR LAPEL SHYLY SHRUG FRAME PURER MOVER SHORE AWFUL BEADY AXIOM AFTER
TENTH ALERT SCONE PRONE MERGE GROOM KNOLL FENCE VILLA ABBEY FIFTY CHICK STRUT
GRAFT WACKY ROOMY KAYAK DANCE DEIGN STATE CROCK MOUND TEACH CRICK SKIMP MUDDY
DELAY KNEAD TONAL BOOZY BLAZE DRONE OTTER ELOPE TITAN SEVER SPIED HEARD APHID
HAPPY LURCH TRAWL STORE TOKEN ABIDE ALLOT OMEGA CREAK HAUNT BULGE SCOWL CLASH
WATER MONEY BATCH THUMP TRUER JOUST ROBIN HENCE EPOXY LYMPH FIELD JIFFY CLOCK
CHARD BARON FLESH IMBUE CROWN PRIDE TRITE BRUNT RECUT SNIDE EXACT TIPSY SILKY
GRAPH GRUFF LEASE WEARY WARTY SPENT BUNNY ABORT TOTAL PEACH FRANK HUSKY ICING
INTRO TODDY SWORE SERUM GAFFE QUACK AMISS APTLY MOURN BRIAR FLOUT AMASS UNITY
```

Output

The first card is "Card 0". Rows and columns count from 0 to 4.

All output will be printed on a single line. First print out which card wins. In case of ties, lower-numbered cards shout "Bingo" first (lower-numbered cards win ties.)

Then print out how they won: "Row *row-number*", "Column *column-number*", "Diag Down" or "Diag Up".

Then print the words (use an asterisk * for the middle square):

- left to right for the row wins
- top to bottom for the column wins
- top to bottom for both diagonal wins

```
Card 2 wins Diag Up UNITY GRATE * AMISS KAPPA
```

Additional Examples

Input 2	Output 2
`files/probAD/cards.1 files/probAD/game.2`	`Card 2 wins Row 1 GROWN SEVEN REPLY WILLY REPEL`

problem 32

Pumpkin Pickup

13 points



"We don't need Two. One and Three can manage the piles." The foreman at the pumpkin patch is proud of his two pumpkin processing robots. Each day, trucks deliver several piles of pumpkins, and the robots carry them from the piles to their processing places. Robot One can carry one pumpkin. Robot Three can carry up to three pumpkins.

The yard is linear, with locations O through T. "One" starts at, and delivers to location O. "Three" starts at, and delivers to location T. The pumpkins piles are locations P, Q, R, and S.

When at O, Robot One chooses the **smallest** pile as its destination. When at T, Robot Three chooses the **largest** pile as its destination. Each chooses closest to its home if there are multiple matching piles.

The robots start at time 0 and take 1 minute to travel to each neighboring location. When either reaches its chosen pile, it picks up the most pumpkins it can carry, sets its destination to home, returns and delivers its load, and picks a new destination. If all piles are empty, the robot stays home. When all piles are empty and both robots are home, the task is done.

If a pile is smaller when the robot arrives at its destination (because the other robot got there first), it collects as many pumpkins as it can carry and returns to its home. It returns without pumpkins if the pile is empty.

Input

The input is one line with 4 numbers (from 1 to 9), the starting count of pumpkins at locations P, Q, R, and S.

```
1 2 3 4
```

Output

Simulate the pumpkin pile pickup. Monitor the pumpkin count and where each robot is at each minute of the collection. **The first 2 lines of the simulation for the above input are below. The next page shows the full output map of the simulation.**

```
 Time   rO       P       Q       R       S       rT
 0       0-o0P   1       2       3       4        0-t0S
```

Print the label row first (Hint: Copy it from a dataset!) There is one space before "Time". The capital letters in the location labels T(ime),rO,P,Q,R,S,rT are 8 characters apart. The lowercase 'r' in 'rO' and 'rT' help identify the home for robots One and Three.

For each minute, print the time in minutes, followed by the count of pumpkins at each location. Also, when a robot is at a location, add 4 characters to show its status:

- a dash (-)
- the robot's lowercase letter 'o' or 't'
- the count of pumpkins it is holding
- its destination (print a space for the destination if it's staying home.)

Print at time 0, and print a new line every minute. Print after any tasks are completed (pickup, dropoff, choosing a new destination). For 2-digit printing, keep the 1's digit below the capital letter in the label.

Output (continued)

Here is the complete output simulation map for the input (**1 2 3 4**) above.

```
Time   rO      P       Q       R       S       rT
0       0-o0P  1       2       3       4        0-t0S
1       0      0-o1O    2       3       1-t3T   0
2       1-o0S  0       2       3       1        3-t0R
3       1      0-o0S    2       3       1-t0R   3
4       1      0       2-o0S   0-t3T   1        3
5       1      0       2       0-o0S   1-t3T   3
6       1      0       2       0       0-o1O   6-t0Q
7       1      0       2       0-o1O   0-t0Q   6
8       1      0       2-o1O   0-t0Q   0        6
9       1      0-o1O   0-t2T   0       0        6
10      2-o0   0       0       0-t2T   0        6
11      2-o0   0       0       0       0-t2T   6
12      2-o0   0       0       0       0        8-t0
```

Discussion

The two robots will never be together at the same location. Complete any pickup or dropoff before either robot chooses a next destination. Then print. For 2-digit printing, keep the 1's digit below the label.

In the example above,
At time 0, One selects P (the smallest pile, with 1), and Three selects S (the largest pile, with 4).
At time 1, One picks up 1 pumpkin, and Three picks up 3 pumpkins (leaving 1 at S).
At time 2, One drops off 1 and selects S (with 1). Three drops off 3 and selects R (with 3).
At time 4, Three picks up all 3 pumpkins from R.
At time 6, One picks up 1, and Three drops off 3 and selects Q (with 2).
At time 9, Three picks up the 2 pumpkins at Q.
At time 10, One drops off 1. It sees all piles empty and stays at home (with no destination).
At time 12, Three drops off the last 2 pumpkins.

problem 33

Hidden Messages 2(026)

14 points

For CodeWars 29, we have brought back "Hidden Messages!" This instruction page is missing, well, instructions. So, you can try to deduce the connection between each character in input and output datasets, or go hunting for instructions to determine how to get from one to the other.



Remember, the instructions are hidden, in plain sight. In the spirit of the several EnCodeWars puzzles this year, be ready to use decoding skills. Instructions on how to decode are already provided. If you look in the right place, you'll find them.

Good luck!

Input

There is one line of input, which can have letters, numbers, punctuation and spaces.

```
This is CodeWars 29! You will search many places to find what you need!
```

Output

The output will be a single integer. How to determine that integer ... is for you to find.

```
1224
```

Discussion

Be sure to run all datasets. Prior years' "Hidden Messages" puzzles may not be helpful.

Additional Examples

Input 2	Output 2
Numbers and Letters: A 1 b 2 C 3 d 4 E 5 f 6 G 7 h 8 I 9 j 10 K 11 l 12 M 13	1034

Input 3	Output 3
You might guess. Hmm. Chances aren't good. Guessing might find 1224, but what about 1034?	111

PN-2-0 HM-3-3 HM-0-3 HM-12-1 HM-15-3 HM-6-4 HM-2-8 HM-0-0 HM-3-1

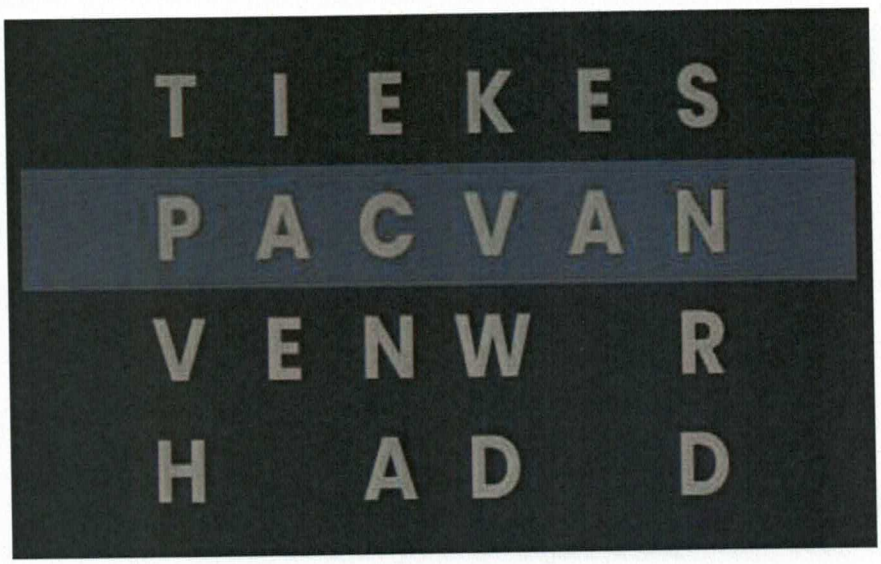
problem 34

Can you crack the Boggle Lock?

15 points

You need to crack a Boggle Lock, this is a lock where each column of letters can be shifted up or down until the highlighted row spells a word in your dictionary.

Now, there are lots of words you can make, but this lock will only open if you enter the minimum number of words that use each letter at least once. Letters can be used multiple times in different words, but each letter needs to be used at least once.



Input

The first line is the length of the word you'll need to form. Then for each letter position, there will be from 2 to 6 possible letters in no particular order.

```
6
TPVH
IAE
ECNA
KVWD
EA
SNRD
```

Output

In the `files/probAC` directory, you have a dictionary of words: `words.txt`. Words created from the lock must be in the dictionary. For your solution, you'll need to consider all the possible words, choosing a letter from each letter position in order. Then you'll need to choose the smallest set of those words required, so that each letter in the input appears in at least one word.

In case of a tie on the fewest number of words, choose the first word alphabetically. Print the words on a single line in alphabetical order, separated by a space.

```
HEAVEN PANDAS TACKED VIEWER
```

Discussion

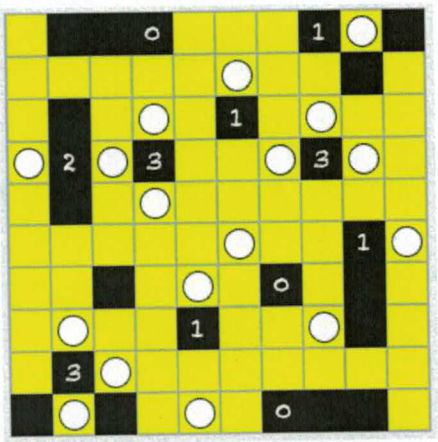
Each of the words will have one letter from each line of the input file and each letter in the input will appear in at least one word. The number of words grows quickly with the number of letters in each row and it will be a challenge to decide which words to keep and still make sure each letter is used at least once.

problem 35

Light The Library

16 points

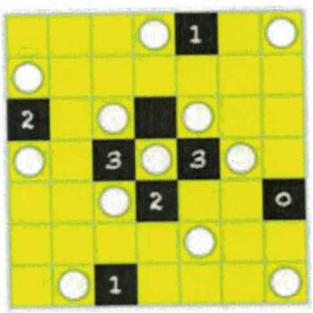
The catacombs of the library have several rooms that haven't been properly lit. We asked illumination expert Lily Lyter to give us some guidance. She returned with maps of each room which show the locations of all the shelves (columns of books) and all the open floor tiles.



On the maps, some of the shelf-columns were marked with numbers. She explained the numbers represent exactly how many lamps must be placed next to those columns to fully light every empty tile.

The lamps only illuminate their aisles and rows (up, down, left, right on the map), not diagonally. Light is blocked by shelf-columns.

But she also had a warning: "The lamps are delicate and may not shine upon each other. Otherwise they will overload with energy and explode." Um, what?



Then she left. So we now need help to interpret the maps and place the lamps so that every empty floor tile is lit.

Input

The first line of input is an integer N (from 4 to 10), the size of the square room. The next N lines show a map of the room. A period (.) is an empty floor tile which needs to be lit. A hash (#) is a shelf-column. A number (0-4) is also a shelf-column, noting exactly how many adjacent empty tiles must hold lamps (not counting diagonally adjacent tiles).

Below are 5 example input maps next to each other, to save vertical space for the problem description. Each dataset contains only one input map.

```
4
.#..
.3..
...#
....

4
..0.
....
..2.
..#.

5
.#...
.0..0
...1.
.....
.0..2

7
....1..
.......
2..#...
..3.3..
...2..0
.......
..1....

10
.##0...1.#
........#.
.#...1....
.2.3...3..
.#........
........1.
..#...0.#.
....1...#.
.3........
#.#...0##.
```

Output

Reprint the original map, replacing a period with an asterisk (*) where each lamp must be placed.

Below are the 5 output maps corresponding to the input maps above.

```
.#.*
*3*.
.*.#
...*

..0.
*...
.*2*
..#.

*#.*.
.0..0
..*1.
....*
.0.*2

...*1.*
*......
2.*#*..
*.3*3*.
..*2..0
....*..
.*1...*

.##0...1*#
.....*..#.
.#.*.1.*..
*2*3..*3*.
.#.*......
.....*..1*
..#.*.0.#.
.*..1..*#.
.3*.......
#*#.*.0##.
```

Discussion

All empty floor tiles must be lit by a lamp in the same aisle, or row, or both; or by a lamp on the tile. A floor tile may be lit by more than one lamp, as long as the lamps do not light each other.

It may help to think of each lamp like a chess rook, able to attack its row and column, and no rook may attack another, and all floor tiles must be attacked or occupied by a rook.

Each dataset has only one answer.

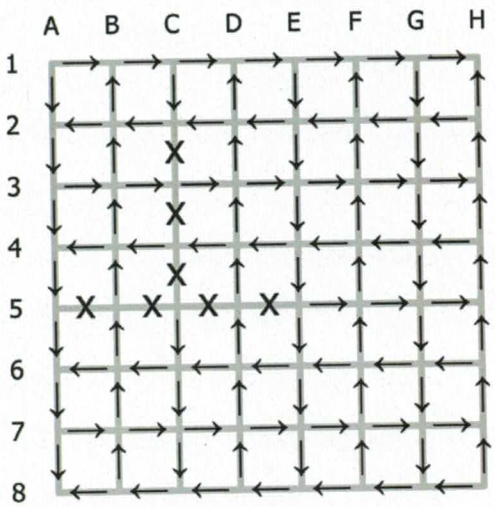
problem 36

No Left Turns

18 points

You need to write the code to provide directions for a robo delivery service drone for a downtown area.

North/south streets are named with Letters and east/west streets are named with Numbers. Intersection A-1 is at the top left (northwest) corner. The area has alternating one way streets. "A" is southbound, "B" is northbound, etc. "1" is eastbound, "2" is westbound, etc. Your robo drone is geofenced and can't leave the street grid.



Input

The first line of input gives you the ranges of the street letters and numbers, starting with A and 1 (the highest will be H and 8). Next comes any closure notices, in the form "CLOSURE Street Start Stop". Closure on a Letter street means no north/south traffic. On a number street, it means no east/west traffic. Intersections are always open (you can cross a street that is closed.) Then come routing requests in the form "ROUTE AllowedTurns IntersectionStart IntersectionEnd". Allowed turns can be ALL, or only RIGHTS or LEFTS.

```
A H 1 8
CLOSURE C 2 5
CLOSURE 5 A E
ROUTE ALL B 2 G 6
ROUTE RIGHTS B 2 G 6
ROUTE LEFTS G 6 B 2
ROUTE LEFTS A 1 C 5
ROUTE RIGHTS C 4 F 7
```

Output

For each closure line, confirm the closures by printing as below. For each routing line, find the shortest possible route. There may be multiple shortest routes, so report the route with the minimum number of turns. Report the total Blocks, number of turns, and directions for each block you travel. If there is no route, report "No route possible"

```
C is closed between 2 and 5
5 is closed between A and E
Routing from (B and 2) to (G and 6) using all turns: 11 Blocks, 2 turns, route: NEEEEESSSSS
Routing from (B and 2) to (G and 6) using right turns: 11 Blocks, 2 turns, route: NEEEEESSSSS
Routing from (G and 6) to (B and 2) using left turns: 13 Blocks, 3 turns, route: SENNNNNWWWWWW
Routing from (A and 1) to (C and 5) using left turns: No route possible
Routing from (C and 4) to (F and 7) using right turns: 14 Blocks, 5 turns, route: WNEEEEESSSSSWN
```

Discussion

The first move does not count as a turn and can be in any legal one-way direction. Going straight is always an option as long as there is no closure. All turns on the route are required to follow the AllowedTurns rule.

On large grids you need to be careful about how you search to find the shortest route. At each intersection think about how to decide if you turn or not. Once you find a route, you need to check all other possible shorter routes to make sure it's the shortest and you also need to see if there's a route that's the same distance but with fewer turns.